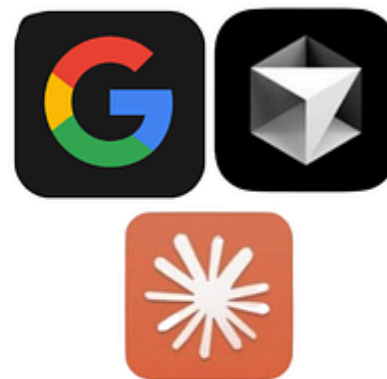
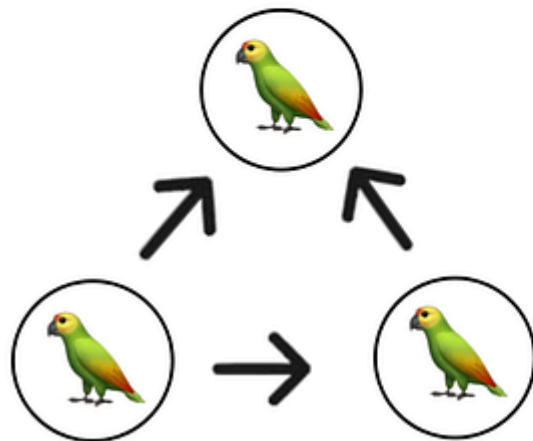


[< Go to the original](#)

LANGCHAIN
A2A-MCP



IS INSANE

A2A + MCP + LangChain = Powerful Agent Communication

In this Story, I have a super quick tutorial showing you how to create a multi-agent chatbot using A2A, MCP, and LangChain to build a...



Follow



Towards AI a11y-light ~16 min read · May 14, 2025 (Updated: May 14, 2025) · Free: No

In this Story, I have a super quick tutorial showing you how to create a multi-agent chatbot using A2A, MCP, and LangChain to build a powerful agent chatbot for your business or personal use.

I recently made a video about the Agent2Agent Protocol and the Model Context Protocol. One of the viewers asked if I could combine MCP and A2A, and I've fulfilled that request. Not only that, but I also powered a chatbot using Langchain, where the agent can analyse and scrape the latest stock news.

As AI technology develops rapidly, two key protocols are reshaping how we build intelligent systems: Google's Agent-to-Agent Protocol (A2A) and Model Context Protocol (MCP). These two protocols represent different dimensions of AI architecture development, but they point to a future together: we are moving from deterministic programming to autonomous collaborative systems.

MCP (Model Context Protocol) is essentially a protocol for **tool access**. It defines a standard way for large language models to interact with various tools, data, and

While A2A (Agent-to-Agent Protocol) focuses on **agent collaboration**, it establishes a way for intelligent agents to discover, communicate and cooperate with each other, allowing different AI systems to work together like human teams.

So, let me give you a quick demo of a live chatbot to show you what I mean.



I will ask the chatbot a question: "What are the current stock prices of Apple and Nvidia?" Feel free to ask any questions you want.

If you look at how the chatbot generates the output, you will see that when I enter a query, the meta-agent analyses the request and intelligently selects appropriate tools.

returns comprehensive metrics including current prices, percentage changes, and key financial indicators.

It may invoke the FinancialNews tool to scrape relevant news from Finviz, properly handling relative URLs by converting them to absolute links. The meta-agent then combines these structured data points with contextual knowledge from the A2A stock market expert agent, which provides professional financial analysis.

I encountered and solved several significant technical challenges during development: networking issues with port conflicts (resolved through my custom `find_available_port` function that dynamically tests port availability), JSON serialization errors with pandas DataFrame objects (addressed by implementing explicit type conversion for all non-serializable objects like timestamps and numpy types), cross-component data transformation (solved by creating wrapper functions that sanitize inputs/outputs between components), error handling (implemented through comprehensive try/except blocks at every level), and dynamic tool selection (managed through LangChain's agent framework)

news, and expert financial context.

Before we start! 🧐

If you like this topic and you want to support me:

1. **Clap** my article 50 times; that will really help me out. 🙌
2. **Follow** me on Medium and subscribe to get my latest article for Free 🎁
3. Join the family — Subscribe to the **YouTube channel**

Let's start coding

Let us now explore step by step and unravel the answer to how to create the A2A and MCP APP. We will install the libraries that support the model. For this, we will do a pip install requirements

Copy

```
pip install requirements
```

We initiate the code by importing classes from

Python A2A is a robust, production-ready library for implementing Google's Agent-to-Agent (A2A) protocol with full support for the Model Context Protocol (MCP). It empowers developers to build collaborative, tool-using AI agents capable of solving complex tasks.

Fastmcp handles all the complex protocol details and server management, so you can focus on building great tools. It's designed to be high-level and Pythonic — in most cases, decorating a function is all you need.

Copy

```
import os
import sys
import socket
import time
import threading
import argparse
import json
from python_a2a import OpenAIA2AServer, run_server, A2AServer, AgentCard, Agent
from python_a2a.langchain import to_langchain_agent, to_langchain_tool
```

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain.agents import initialize_agent, Tool
from langchain.agents import AgentType
import yfinance as yf
import pandas as pd
import json
# Import required libraries
import json
import requests
from bs4 import BeautifulSoup
import re
from urllib.parse import urljoin
```

I created a utility to help automatically find an open port and run a server in the background, which is useful for applications like local testing or web apps. First, I developed the `find_available_port` function to start checking ports from a given number (default is 5000) and try up to 20 consecutive ports.

It does this by attempting to bind a socket to each port; if it fails due to the port being in use, it moves to the next one. If no ports are available, it falls back to a much higher port number to avoid common conflicts.

This lets the server run in the background while the main program continues, and I added a `sleep Time` to make sure the server has time to start properly before other code interacts with it.

Copy

```
def find_available_port(start_port=5000, max_tries=20):
    """Find an available port starting from start_port"""
    for port in range(start_port, start_port + max_tries):
        try:
            # Try to create a socket on the port
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.bind(('localhost', port))
            sock.close()
            return port
        except OSError:
            # Port is already in use, try the next one
            continue

    # If we get here, no ports were available
    print(f"⚠️ Could not find an available port in range {start_port}-{start_port + max_tries}")
    return start_port + 1000 # Try a port well outside the normal range

def run_server_in_thread(server_func, server, **kwargs):
    """Run a server in a background thread"""
    thread = threading.Thread(target=server_func, args=(server,), kwargs=kwargs)
```

return True

I built the `A_parse_arguments` function to handle command-line input for customizing how the script runs, especially for configuring server ports and model behaviour. I used Python's `ArgumentParser` module to create a parser with helpful descriptions, making it easy for users to understand what each option does.

I added arguments like `--a2a-port` and `--mcp-port` to let users manually set the ports for two different servers, though they default to `None`. So the program can auto-select if needed.

I also included options like `--model` to specify which OpenAI model to use (defaulting to `gpt-4o`), and `--temperature` to control how creative the responses are, with a default of `0.0` for deterministic outputs.

Copy

```
def parse_arguments():  
    """Parse command line arguments"""  
    parser = argparse.ArgumentParser(description="A2A + MCP + LangChain Stock M  
    parser.add_argument(  
        "--a2a-port", type=int, default=None,
```

```
parser.add_argument(
    "--mcp-port", type=int, default=None,
    help="Port to run the MCP server on (default: auto-select)"
)
parser.add_argument(
    "--model", type=str, default="gpt-4o",
    help="OpenAI model to use (default: gpt-4o)"
)
parser.add_argument(
    "--temperature", type=float, default=0.0,
    help="Temperature for generation (default: 0.0)"
)
return parser.parse_args()
```

Then I developed an AI server for stock market analysis using Openai's API and a modular multi-agent system.

First, I checked if the user provided custom ports via the command line; if not, I used `find_available_port` to automatically choose one for each server, with different ranges to prevent port conflicts. Then, I printed the selected ports to help with debugging.

I moved on to **Step 1**, where I created an Openai-powered A2A (Agent-to-Agent) server. To define its role and capabilities, I made an `AgentCard` with a name,

After that, I initialized the `OpenAIA2AServer` using the OpenAI model type, temperature, and a detailed system prompt instructing the AI to act as a financial expert.

Copy

```
def main():
    """Main function"""
    # Check API key
    if not check_api_key():
        return 1

    # Parse arguments
    args = parse_arguments() # Find available ports - use different ranges to av
    a2a_port = args.a2a_port or find_available_port(5000, 20)
    mcp_port = args.mcp_port or find_available_port(7000, 20) # Try higher port

    print(f"🔍 A2A server port: {a2a_port}")
    print(f"🔍 MCP server port: {mcp_port}")

    # Step 1: Create the OpenAI-powered A2A server for Stock Market Expertise
    print(f"\n📝 Step 1: Creating OpenAI-Powered A2A Server")

    # Create an Agent Card for our OpenAI-powered agent
    agent_card = AgentCard(
        name="Stock Market Expert",
```

```

version= 1.0.0 ,
skills=[
    AgentSkill(
        name="Market Analysis",
        description="Analyzing market trends, stock performance, and market sentiment",
        examples=["What's the current market sentiment?", "How do interest rates affect the market?"]
    ),
    AgentSkill(
        name="Investment Strategies",
        description="Providing information about various investment approaches and risk management",
        examples=["What is dollar cost averaging?", "How should I diversify my portfolio?"]
    ),
    AgentSkill(
        name="Company Analysis",
        description="Analyzing specific companies, their fundamentals, and growth prospects",
        examples=["What are key metrics to evaluate a tech company?", "How do I analyze a company's financial statements?"]
    )
]

# Create the OpenAI server
openai_server = OpenAIA2AServer(
    api_key=os.environ["OPENAI_API_KEY"],
    model=args.model,
    temperature=args.temperature,
    system_prompt="You are a stock market and financial analysis expert. Provide accurate and concise information."
)

```

scalable way.

I defined a new class, `StockMarketExpert` which inherits from `A2AServer` and overrides the `handle_message` method to delegate incoming messages directly to the OpenAI backend. This makes the A2A interface compatible with the broader multi-agent ecosystem.

After wrapping the OpenAI server and agent card into this class, I instantiated it as `stock_agent`. To get the server running in the background without blocking the main program, I used the `run_server_in_thread` function and passed in `run_a2a_server`, which binds the server to the appropriate host and port.

Copy

```
# Wrap it in a standard A2A server for proper handling
class StockMarketExpert(A2AServer):
    def __init__(self, openai_server, agent_card):
        super().__init__(agent_card=agent_card)
        self.openai_server = openai_server

    def handle_message(self, message):
        """Handle incoming messages by delegating to OpenAI server"""
        return self.openai_server.handle_message(message)
```

```
# Start the A2A server in a background thread
a2a_server_url = f"http://localhost:{a2a_port}"
print(f"\nStarting A2A server on {a2a_server_url}...")

def run_a2a_server(server, host="0.0.0.0", port=a2a_port):
    """Run the A2A server"""
    run_server(server, host=host, port=port)

a2a_thread = run_server_in_thread(run_a2a_server, stock_agent)

# Step 2: Create MCP Server with Finance Tools
print("\n📝 Step 2: Creating MCP Server with Finance Tools")

# Create MCP server with tools
mcp_server = FastMCP(
    name="Finance Tools",
    description="Advanced tools for stock market analysis"
)
```

I created a stock data fetcher MCP tool to allow users to retrieve financial data. I started by designing a function `stock_data` that can accept either a keyword argument or a direct string, which it then parses to extract ticker symbols—whether they're comma-separated or embedded in plain English (like "apple" or "Tesla"). To handle this,

input to customise the data period, and I used the `yfinance` library to pull historical data and detailed company info.

I calculated key statistics for each stock, like price change, percent change, average volume, market cap, and P/E ratio. All results are structured in a detailed JSON summary, and the function is wrapped with exception handling to ensure robustness and informative error reporting

Copy

```
# Stock Data Fetcher Tool - Enhanced
@mcp_server.tool(
    name="stock_data",
    description="Fetch stock data for one or more ticker symbols"
)
def stock_data(input_str=None, **kwargs):
    """Fetch stock data using yfinance with enhanced parsing."""
    try:
        # Handle both positional and keyword arguments
        if 'input' in kwargs:
            input_str = kwargs['input']

        # Make sure we have a string
        if input_str is None:
            return {"text": "Error: No ticker symbol provided"}
```



```
# Extract tickers - support multiple formats
tickers = []

# Check for comma-separated tickers
if ',' in input_str:
    tickers = [t.strip().upper() for t in input_str.split(',') if t]
else:
    # Extract words that look like tickers (1-5 letters)
    import re
    tickers = [word.upper() for word in re.findall(r'\b[A-Za-z]{1,5}', input_str)]

# Map common stock names to tickers if no explicit tickers found
if not tickers:
    common_stocks = {
        'apple': 'AAPL', 'microsoft': 'MSFT', 'google': 'GOOGL',
        'amazon': 'AMZN', 'tesla': 'TSLA', 'nvidia': 'NVDA'
    }
    for name, ticker in common_stocks.items():
        if name.lower() in input_str.lower():
            tickers.append(ticker)

if not tickers:
    return {"text": "No valid ticker symbols found in input"}

# Default parameters
period = "1mo"
interval = "1d"_sy

# Very simple parameter extraction
```

```
def get_data(ticker_symbol, interval, period):  
    # Get data for the specified ticker and period  
    period = "1wk"  
  
    # Import libraries  
    import yfinance as yf  
    import pandas as pd  
    import json  
  
    # Process each ticker  
    results = {}  
  
    for ticker_symbol in tickers:  
        try:  
            # Fetch data  
            ticker = yf.Ticker(ticker_symbol)  
            hist = ticker.history(period=period, interval=interval)  
  
            if hist.empty:  
                results[ticker_symbol] = {"error": f"No data found for {ticker_symbol}"}  
                continue  
  
            # Get more comprehensive info  
            info = ticker.info  
            company_name = info.get('shortName', ticker_symbol)  
            sector = info.get('sector', 'Unknown')  
            industry = info.get('industry', 'Unknown')  
  
            # Enhanced summary with more metrics  
            latest = hist.iloc[-1]
```

```
percent_change = (price_change / float(earliest['close']))

# Calculate some key statistics
high_52week = info.get('fiftyTwoWeekHigh', 'Unknown')
low_52week = info.get('fiftyTwoWeekLow', 'Unknown')
avg_volume = info.get('averageVolume', 'Unknown')
market_cap = info.get('marketCap', 'Unknown')
pe_ratio = info.get('trailingPE', 'Unknown')

# Create a more comprehensive summary
summary = {
    "ticker": ticker_symbol,
    "company_name": company_name,
    "sector": sector,
    "industry": industry,
    "latest_price": float(latest['Close']),
    "price_change": float(price_change),
    "percent_change": float(percent_change),
    "52_week_high": high_52week,
    "52_week_low": low_52week,
    "average_volume": avg_volume,
    "market_cap": market_cap,
    "pe_ratio": pe_ratio,
    "period": period,
    "interval": interval,
    "data_points": len(hist)
}

results[ticker_symbol] = summary
```

```
results[ticker_symbol] = { 'error': 'Error processing ticker'

    return {"text": json.dumps(results)}
except Exception as e:
    import traceback
    error_details = traceback.format_exc()
    return {"text": f"Error: {str(e)}\nDetails: {error_details}"}
```

I developed A web_scraper MCP tool to fetch financial news and company data using a flexible input system that handles ticker symbols, URLs, and general topics.

First, I set up a tool function with a descriptive name and an input handler that checks both positional and keyword arguments. After ensuring the input is a valid string, I use regular expressions to identify whether it's a ticker symbol, URL, or topic.

For ticker symbols like "AAPL," the agent sends a request to Finviz, scrapes the HTML with BeautifulSoup, and extracts the latest five news headlines and company snapshot details. If the input is a URL, it simply returns the link with a message. If it's a general topic, it prompts the user to use a web search instead.

[Copy](#)

```
        name="web_scraper",
        description="Get financial news and information"
    )
def web_scraper(input_str=None, **kwargs):
    """Get financial news using web search."""
    try:
        # Handle both positional and keyword arguments
        if 'input' in kwargs:
            input_str = kwargs['input']

        # Make sure we have a string
        if input_str is None:
            return {"text": "Error: No input provided"}

        input_str = str(input_str).strip()

        # Import required libraries
        import json
        import requests
        from bs4 import BeautifulSoup
        import re
        from urllib.parse import urljoin

        # Determine if this is a ticker, URL, or topic
        if re.match(r'^[A-Za-z]{1,5}$', input_str):
            # It's a ticker symbol – get stock news
            ticker = input_str.upper()

            # Use a simple approach with Finviz
```

```
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, 'html.parser')

# Get news items
news_table = soup.find('table', {'id': 'news-table'})
news_items = []

if news_table:
    for row in news_table.find_all('tr')[:5]:
        cells = row.find_all('td')
        if len(cells) >= 2:
            date_cell = cells[0]
            title_cell = cells[1]

            link = title_cell.find('a')
            if link:
                news_link = link['href']
                # Fix relative URLs
                if not news_link.startswith('http'):
                    news_link = urljoin(url, news_link)

            news_items.append({
                "title": link.text.strip(),
                "link": news_link,
                "date": date_cell.text.strip()
            })

# Try to get some company details from Finviz as well
```

```
11 snapshot_table.  
    rows = snapshot_table.find_all('tr')  
    for row in rows:  
        cells = row.find_all('td')  
        for i in range(0, len(cells), 2):  
            if i+1 < len(cells):  
                key = cells[i].text.strip()  
                value = cells[i+1].text.strip()  
                if key and value:  
                    company_details[key] = value  
  
    return {"text": json.dumps({  
        "ticker": ticker,  
        "news_items": news_items,  
        "company_details": company_details  
    })}  
  
elif input_str.startswith('http'):  
    # It's a URL – just return basic info  
    return {"text": json.dumps({  
        "url": input_str,  
        "message": "URL processing is simplified in this version."  
    })}  
else:  
    # Treat as a topic  
    topic = input_str.replace("topic:", "").strip()  
  
    return {"text": json.dumps({  
        "topic": topic,
```

```
except Exception as e:
    import traceback
    error_details = traceback.format_exc()
    return {"text": f"Error: {str(e)}\nDetails: {error_details}"}
```

I created this system to launch an MCP server in the background, convert its tools and an A2A agent into LangChain-compatible components, and test everything in a streamlined workflow.

I have initialised the MCP server on a specified port using a background thread, then paused briefly to ensure the server is fully online. If the initial port fails, the system dynamically finds a new open port and restarts the server.

Once the MCP server is verified as running, I convert the A2A agent and the MCP tools, like the `stock_data` and `web_scraper` MCP tool into LangChain tools using helper functions.

After successful conversion, I run tests on each component: first querying the A2A agent for financial insights, and then invoking the MCP tools with a sample stock ticker like "AAPL" to confirm they return accurate data.


```
# Start the MCP server in a background thread
mcp_server_url = f"http://localhost:{mcp_port}"
print(f"\nStarting MCP server on {mcp_server_url}...")

def run_mcp_server(server, host="0.0.0.0", port=mcp_port):
    """Run the MCP server"""
    server.run(host=host, port=port)

mcp_thread = run_server_in_thread(run_mcp_server, mcp_server)

# Wait a bit longer for the MCP server to start
print("\nWaiting for servers to initialize...")
time.sleep(5)

# Check if MCP server is actually running
mcp_server_running = False
try:
    import requests
    response = requests.get(f"{mcp_server_url}/tools")
    if response.status_code == 200:
        mcp_server_running = True
except:
    pass

if not mcp_server_running:
    print(f"❌ MCP server failed to start on port {mcp_port}.")
    print("Let's try a different port...")

    # Try a different port
```

```
print(f"Trying MCP server on {mcp_server_url}")

# Start the MCP server in a background thread with new port
mcp_thread = run_server_in_thread(run_mcp_server, mcp_server, port=mcp_
time.sleep(5)

# Step 3: Convert A2A agent to LangChain
print("\n📝 Step 3: Converting A2A Agent to LangChain")

try:
    langchain_agent = to_langchain_agent(a2a_server_url)
    print("✅ Successfully converted A2A agent to LangChain")
except Exception as e:
    print(f"❌ Error converting A2A agent to LangChain: {e}")
    return 1

# Step 4: Convert MCP tools to LangChain tools
print("\n📝 Step 4: Converting MCP Tools to LangChain")

try:
    stock_data_tool = to_langchain_tool(mcp_server_url, "stock_data")
    web_scraper_tool = to_langchain_tool(mcp_server_url, "web_scraper")
    print("✅ Successfully converted MCP tools to LangChain")
except Exception as e:
    print(f"❌ Error converting MCP tools to LangChain: {e}")
    print("\nContinuing with only the A2A agent...")
    stock_data_tool = None
    web_scraper_tool = None
```

```
# Test A2A agent via LangChain
try:
    print("\nTesting A2A-based LangChain agent:")
    result = langchain_agent.invoke("What are some key metrics to evaluate")
    print(f"A2A Agent Response: {result.get('output', '')}")
except Exception as e:
    print(f"❌ Error using A2A-based LangChain agent: {e}")

# Test MCP tools via LangChain if available
if stock_data_tool and web_scraper_tool:
    try:
        print("\nTesting MCP-based LangChain tools:")

        print("\n1. Stock Data Tool:")
        stock_result = stock_data_tool.invoke("AAPL")
        print(f"Stock Data Tool Response: {stock_result[:500]}..." # Truncated

        print("\n2. Web Scraper Tool:")
        web_result = web_scraper_tool.invoke("AAPL")
        print(f"Web Scraper Tool Response: {web_result[:500]}..." # Truncated

    except Exception as e:
        print(f"❌ Error using MCP-based LangChain tools: {e}")
        import traceback
        traceback.print_exc()
```

configurable parameters like model and temperature. Then I wrapped the core logic of each tool — like the A2A-based stock expert, the MCP stock data retriever, and the web scraper — into callable Python functions that safely handle input and error cases.

I registered each of these functions as LangChain `Tool` objects with clear names and descriptions, making them accessible to the agent. Finally, I used `initialize_agent` to combined all these tools into a single meta-agent that uses OpenAI Functions to intelligently choose which tool to call based on the user's query

Copy

```
# Step 6: Creating a Meta-Agent with available components
print("\n📝 Step 6: Creating Meta-Agent with Available Tools")

try:
    # Create an LLM for the meta-agent
    llm = ChatOpenAI(model=args.model, temperature=args.temperature)

    # Create wrapper functions for the LangChain tools
    def ask_stock_expert(query):
        """Ask the stock market expert agent a question."""
        try:
```

```
except Exception as e:
    return f"Error querying stock expert: {str(e)}"

# Create tools list starting with the A2A agent
tools = [
    Tool(
        name="StockExpert",
        func=ask_stock_expert,
        description="Ask the stock market expert questions about invest
    )
]

# Add MCP tools if available
if stock_data_tool:
    def get_stock_data(ticker_info):
        """Get stock data for analysis."""
        try:
            # Ensure ticker_info is a string
            if ticker_info is None:
                return "Error: No ticker symbol provided"
            if not isinstance(ticker_info, str):
                ticker_info = str(ticker_info)

            result = stock_data_tool.invoke(ticker_info)
            return result
        except Exception as e:
            return f"Error getting stock data: {str(e)}"

    tools.append(Tool(
```

```
description="Get historical stock data. Input can be one or more tickers."
    ))

if web_scraper_tool:
    def get_financial_news(query):
        """Get financial news and information."""
        try:
            # Ensure query is a string
            if query is None:
                return "Error: No query provided"
            if not isinstance(query, str):
                query = str(query)

            result = web_scraper_tool.invoke(query)
            return result
        except Exception as e:
            return f"Error getting financial news: {str(e)}"

tools.append(Tool(
    name="FinancialNews",
    func=get_financial_news,
    description="Get financial news. Input can be a ticker symbol,
    ))

# Create the meta-agent
meta_agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.OPENAI_FUNCTIONS,
```

I developed this final part of the AI agent to test a fully integrated meta-agent that combines all available tools, like the A2A agent and MCP tools, under one unified LangChain interface.

I queried, *"What are the current stock prices of Apple and Nvidia?"* to the system, `meta_agent` which intelligently delegates tasks to the appropriate tools based on the query content.

After printing the meta-agent's response, I ensure the whole setup keeps running smoothly by entering a loop that waits for user interruption, allowing the background servers to stay active.

Copy

```
# Test the meta-agent
print("\nTesting Meta-Agent with available tools:")

test_query = "What are the current stock prices of Apple and Nvidia?"

print(f"\nQuery: {test_query}")
meta_result = meta_agent.invoke(test_query)
print(f"\nMeta-Agent Response: {meta_result}")
```

```
print(f"❌ Error creating or using meta-agent: {e}")
import traceback
traceback.print_exc()

# Keep the servers running until user interrupts
print("\n✅ Integration successful!")
print("Press Ctrl+C to stop the servers and exit")

try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    print("\nExiting...")

return 0

if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        print("\nProgram interrupted by user")
        sys.exit(0)
```

MCP and A2A represent two key dimensions of AI system construction — one for tool integration and one for agent collaboration. Together, they mark a fundamental shift in the software development paradigm: from explicit programming to descriptive, autonomous, and collaborative systems.

but can think, adapt, and collaborate autonomously to complete complex tasks. We are no longer programming software, but collaborating with intelligent systems.

This is not just an evolution of AI architecture, but a revolution in the entire way of software development.

 *I am an AI Generative expert! If you want to collaborate on a project, drop an [inquiry here](#) or Book a [1-on-1 Consulting Call With Me](#).*

 *Feel free to check out my other articles:*

Build Anything With A2A Agent — Here's How (Part 1)

Recently, I moved from Taiwan to North Carolina. During a transfer flight, I had some time to scroll through YouTube...

medium.com

agent chatbot using LangChain, MCP, RAG...

medium.com

LangGraph + MCP + Ollama: The Key To Powerful Agentic AI

In this story, I have a super quick tutorial showing you how to create a multi-agent chatbot using LangGraph, MCP, and...

medium.com

[#data-science](#)

[#machine-learning](#)

[#artificial-intelligence](#)

[#technology](#)

[#programming](#)